

Rapport de Conception

Biljana Djordjevic

Noa Pistone

Introduction générale du projet

Dans le cadre de ce projet, nous avons développé trois jeux web utilisant différentes techniques de rendu : le DOM (structure de la page manipulée en JavaScript), le Canvas (zone de dessin 2D contrôlée par le code) et Babylon.js (moteur de rendu 3D dans le navigateur). Nous devons réaliser trois jeux en utilisant différentes méthodes de développement. Les trois jeux ont été regroupés dans un site web unique, mis en ligne et accessible via navigateur.

Lien du site hébergé : <https://l3-miage-web-home-page.vercel.app>

Présentation générale des trois jeux

> **NEON ESCAPE** (Canvas). Le joueur doit atteindre la sortie le plus rapidement possible tout en évitant les obstacles et en collectant des pièces afin d'augmenter son score.

> **Le Labyrinthe** (Babylon.js). Le joueur doit trouver la sortie à travers différents niveaux. Il peut collecter des objets pour gagner des points et doit éviter les ennemis, qui lui font perdre des points de vie.

> **Puissance 4** (DOM) est une version revisitée du jeu classique. Des pions spéciaux ont été ajoutés, notamment un pion capable de déclencher une explosion affectant les pions voisins.

NEON ESCAPE

Fonctionnement

Le jeu se joue uniquement avec les flèches du clavier. Le joueur se déplace pour atteindre la sortie tout en évitant les obstacles. Les pièces rapportent 10 points chacune. Les niveaux sont chronométrés. Le joueur dispose de 5 vies et perd une vie en cas de contact avec un obstacle ou un ennemi.

Difficultés rencontrées

Les principales difficultés rencontrées concernent la gestion des événements, le mouvement des obstacles, l'organisation des états du jeu et la suppression dynamique de certains éléments. Ces problèmes ont été résolus par une meilleure structuration du code, la mise en place de limites de déplacement et la séparation des différents états du jeu en modules distincts. Les détails de ces problèmes et de leurs solutions sont expliqués plus en profondeur dans le README du jeu.

LE LABYRINTHE

Fonctionnement

Le déplacement du joueur se fait avec les touches ZQSD. La souris permet d'orienter la vue afin d'explorer l'environnement. Différents objets sont répartis dans le labyrinthe : certains permettent de restaurer des points de vie, d'autres rapportent des points, un objet augmente la vitesse du joueur, et un autre indique le chemin vers la sortie. Le jeu comporte un timer et 7 niveaux de difficulté progressive, avec un nombre d'ennemis croissant.

Difficultés rencontrées

Plusieurs difficultés ont été rencontrées lors du développement, notamment la gestion de l'algorithme A*, les collisions, la caméra, la diversité des ennemis ainsi que l'importation et la gestion des modèles 3D et de leurs animations. Ces problématiques ont été résolues grâce à différentes optimisations (gestion du blocage des ennemis, recalibrage des hitbox, amélioration de la caméra et mise en place d'un système de

chargement et d'animation adapté). Les détails techniques complets de ces solutions sont expliqués dans le README du jeu.

PUISSANCE 4

Fonctionnement

Cette version revisite le Puissance 4 classique en ajoutant des pions spéciaux. Certains permettent de faire exploser les pions voisins, un autre provoque une explosion en diagonale, et un bonus “flash” rend temporairement tous les pions invisibles (blancs) pour l'adversaire. Ces mécaniques ajoutent une dimension stratégique au jeu traditionnel.

Difficultés rencontrées

Plusieurs difficultés ont été rencontrées, notamment la gestion de la gravité après les explosions, la synchronisation entre les animations et la logique du jeu, ainsi que la gestion des pouvoirs spéciaux et des effets de tour. Ces problèmes ont été résolus par la mise en place d'un système de gravité permettant de réorganiser la grille après une explosion, l'utilisation de fonctions asynchrones pour synchroniser les animations CSS, ainsi qu'une unification du système de sélection des bonus pour éviter les conflits d'états. Le mode “Flash” a également été ajusté afin de s'activer et se désactiver correctement lors des changements de tour. Les détails techniques complets sont expliqués dans le README du jeu.

Organisation des fichiers du projet

- Organisation du *repository* :

Le projet est organisé en plusieurs dossiers afin de séparer clairement les différents jeux. Cela permet d'éviter les conflits de code et de rendre le projet plus lisible.

- ***index.html*** : pages principales du site regroupant les trois jeux
- ***jeubabylon*** : dossier pour le jeu Babylon.js
- ***jeucanva*** : dossier pour le jeu Canvas
- ***jeudom*** : dossier pour le jeu DOM

- Organisation du dossier *jeucanva* :

Le jeu Canvas est organisé en plusieurs dossiers afin de structurer le code par fonctionnalités :

- **css** : styles du jeu
- **js/main.js** : point d'entrée du jeu
- **js/game** : logique principale (gestion du jeu, niveaux, collisions, événements)
- **js/objetsJeu** : classes des éléments du jeu (joueur, ennemis, obstacles...)
- **js/etats** : gestion des différents états du jeu (menu, game over, transition...)
- **js/utils** : fonctions utilitaires et composants réutilisables

Tout le code est dans **js/** découpé en quatre sous-dossiers logiques.

objetsJeu/ contient une classe par entité du jeu : le joueur, les obstacles, l'ennemi, les pièces, la sortie. Chaque entité sait se dessiner et se comporter de façon autonome. Cela évite un fichier unique de plusieurs centaines de lignes impossible à maintenir.

etats/ regroupe les différentes phases du jeu : Menu, Jeu en cours, Transition entre niveaux, JeuTermine et GameOver. Chaque état est isolé dans son propre fichier et prend la main quand c'est son tour. Cette organisation évite les conditions imbriquées dans la boucle principale et rend les transitions plus claires.

game/ contient la logique centrale : **Jeux.js** orchestre la boucle de jeu, collisions.js gère uniquement la détection des collisions, niveaux.js définit la configuration de chaque niveau, et ecouteurs.js centralise tous les événements clavier et souris en un seul endroit.

utils/ regroupe des éléments réutilisables dans tout le projet : la classe **Bouton.js** pour créer des boutons Canvas sans dupliquer le code, et **utils.js** pour des fonctions génériques partagées.

- Organisation du dossier *jeubabylon* :

Le jeu Babylon.js est organisé en deux grandes parties :

- **public/assets** : contient tous les modèles 3D et ressources externes (personnage, monstres, textures)
- **src** : contient tout le code du jeu

Dans le dossier **src**, le code est séparé par fonctionnalités :

- fichiers principaux du jeu (main.js, scene.js, labyrinthe.js)
- gestion du joueur (joueur.js, playerState.js)
- gestion des ennemis (enemy.js, enemyManager.js)
- gestion des objets et items (items.js, itemsManager.js)
- logique de jeu (gameManager.js, story.js)
- systèmes annexes (score, vie, timer, A* pour les déplacements)

À l'intérieur de **src/**, chaque fichier a une responsabilité unique : **enemy.js** gère un seul ennemi, **enemyManager.js** gère la liste des ennemis et leur interaction avec le joueur, **scoreManager.js** s'occupe uniquement du score et des items, **vieManager.js** uniquement de la barre de vie. Cette séparation évite d'avoir des fichiers trop longs et difficiles à relire, et permet de modifier une fonctionnalité sans risquer d'en casser une autre. Les assets sont eux-mêmes organisés en sous-dossiers par type (**textures/**, **skybox/**, **items/**, **icons/**) pour retrouver facilement un fichier sans parcourir une longue liste.

- Organisation du dossier *jeudom*:

Le jeu DOM possède une structure plus simple, adaptée à un projet de taille plus réduite :

- **index.html** : structure de la page du jeu
- **style.css** : mise en forme et animations visuelles
- **main.js** : point d'entrée du jeu
- **Game.js** : logique principale du Puissance 4 (règles, tours, bonus, vérification de victoire)
- **UI.js** : gestion de l'affichage et des interactions avec l'interface

Le projet est volontairement simple et peu découpé car le jeu lui-même est moins complexe que les deux autres. Trois fichiers JavaScript suffisent à tout gérer.

Game.js contient uniquement la logique pure du jeu : la grille, les règles, la détection de victoire, le système de bonus et la gravité. Ce fichier ne touche jamais au DOM ni à l'affichage. **UI.js** fait l'inverse : il s'occupe exclusivement de l'affichage, des animations, des particules et de la mise à jour des scores à l'écran. Il ne contient aucune règle de jeu. **main.js** fait le lien entre les deux : il écoute les clics du joueur, appelle **Game.js** pour mettre à jour l'état, puis appelle **UI.js** pour rafraîchir l'affichage.

Cette séparation entre logique et affichage est volontaire. Si on veut changer l'apparence du jeu, on ne touche qu'à **UI.js**. Si on veut modifier les règles ou ajouter un bonus, on ne touche qu'à **Game.js**. Les deux parties du code ne se mélangent jamais, ce qui rend le projet plus facile à faire évoluer.

Conclusion

Ce projet nous a permis de concevoir et développer trois jeux web en utilisant différentes méthodes de rendu : le DOM, le Canvas et Babylon.js. Il nous a également permis de mieux comprendre l'organisation d'un projet web, la séparation des responsabilités dans le code ainsi que les problématiques liées au gameplay, aux animations et à la gestion des interactions. Enfin, la mise en ligne des jeux sur un site unique a permis d'aboutir à un projet complet et fonctionnel.